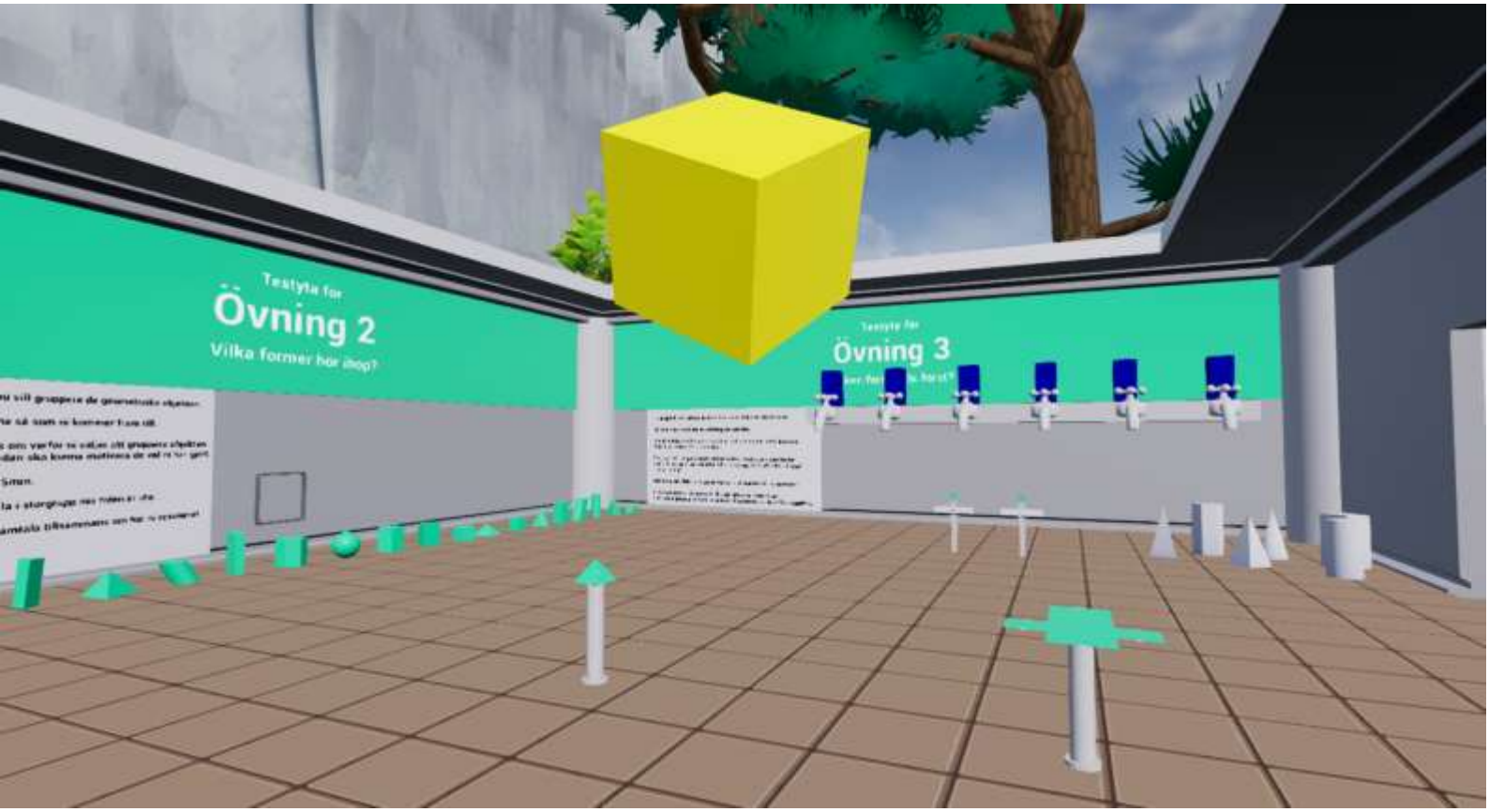
# Documentation of the immersive geometry laboratory - VäXR




VäXR is an Unreal Engine project available on GitHub under MIT license.

https://github.com/Petter-Wannerberg/VaXR

This is the official documentation for the VäXR application. This document was created in collaboration with AI (Chat GPT 5.2). All information has been examined to reliably represent the view of the developers.

**1. Project overview** ..... **3**
*Background and purpose* ..... *3*
*Feature overview* ..... *3*
*High-Level Architecture* ..... *4*
**2. Recommended skills** ..... **7**
**3. Development environment and tooling** ..... **10**
*Engine version and core setup* ..... *10*
*Required plugins and dependencies* ..... *10*
*Android build configuration and SDK setup* ..... *10*
*Version control and submodules* ..... *11*
*Backend build and deployment tooling* ..... *11*
*Development workflow considerations* ..... *11*
**4. Description of back-end systems** ..... **13**
*Multiuser architecture – how it works* ..... *13*
*Server function, architecture and setup* ..... *13*
*Session ("lektion") management and code system* ..... *15*
*Administrative interface and database management* ..... *15*
**5. Description of real-time systems** ..... **17**
*VR player controller (player avatar)* ..... *17*
*Teleportation (NavMesh)* ..... *18*
*General interaction (grabbing and throwing)* ..... *19*
*Control panels and teacher tools* ..... *20*
**6. Description of exercises** ..... **22**
*Toggle Button* ..... *22*
*Exercise 1 - Build 3D figures out of 2D sides* ..... *22*
*Exercise 2 – Sort shapes* ..... *24*
*Exercise 3 - Fill shapes* ..... *26*
**7. Level Structure & World Logic** ..... **28**
*Persistent level as gathering spot* ..... *28*
*Streamed sublevels as exercise rooms* ..... *28*
*Distributed exercise logic* ..... *28*
*Centralized grouping and control logic* ..... *29*
*Room lifecycle and synchronization* ..... *29*
*Architectural flexibility* ..... *29*

RI.
SE

*Details of Implementation of centralized logic* ..... *30*

**8. Optimization** ..... **32**

*LOD system* ..... *32*

*Distance occlusion culling and VR constraints* ..... *32*

*Post processing and anti-aliasing* ..... *32*

*Rendering settings and lighting strategy* ..... *33*

*Known performance drains* ..... *33*

**9. Known issues and technical debt** ..... **34**

**10. Publishing workflow** ..... **37**

**11. Contacts and sources** ..... **40**

*Project ownership and ongoing responsibility* ..... *40*

*Primary and secondary contacts* ..... *40*

*Participating organizations and contributors* ..... *40*

*External libraries, tools, and resources* ..... *41*

RI.
SE

# 1. Project overview

This chapter introduces the VäXR immersive geometry laboratory, with a short background and purpose of the project, its educational foundation in mathematical didactic research, and its intended users. It gives a high-level overview of the application's main features, user roles, and capabilities, as well as the overall technical context in which the project operates. See this chapter as an initial orientation to grasp what the project contains. Each part is explained in further detail in the subsequent chapters.

## Background and purpose

VäXR was developed as an immersive geometry laboratory designed to support the teaching and learning of mathematics through embodied interaction and collaboration. The application is grounded in mathematical didactic research, with a particular focus on helping students develop conceptual understanding by actively constructing, manipulating, and discussing geometric forms rather than passively observing them. Instead of presenting geometry as static diagrams on a flat surface, VäXR places students inside a shared spatial environment where shapes can be built, compared, sorted, and explored together.

The primary users of the application are students working in small groups and teachers facilitating structured lessons. Students engage directly with geometric tools and exercises, often in pairs, while communicating and reasoning together. Teachers guide the process, introduce tasks, manage group work, and support reflection and discussion. The overall purpose of the project is to create a learning environment where spatial reasoning, collaboration, and dialogue become central components of geometry education.

## Feature overview

This list offers an overview of the features and the application.

- **Single user and multiuser platform**
  VäXR can be used either as a standalone exploratory experience or as a structured multiuser lesson. In single-user mode, a player can freely explore the environment and interact with the available learning tools. In multiuser mode, one teacher and up to eight students connect to a shared lesson session where collaboration and guided instruction take place in real time.

- **Player capabilities (interaction, laser-pointer, teleportation)**
  All users can move through the environment using teleport-based locomotion and interact with objects using VR hand controls. Players can grab, move, rotate, assemble, and manipulate learning objects. A laser pointer enables pointing at objects or remotely grabbing items from a distance, supporting both communication and interaction.

- **Student management (grouping, moving to classrooms, gathering etc.)**
  During a lesson, students are organized into small groups, typically pairs, and assigned to dedicated exercise rooms. The teacher can send groups out to their working areas,

visit them individually, gather everyone back to the central space (*Agora*), and temporarily restrict movement to regain attention.

- **Introduction of exercises and lesson navigation**
  Exercises are introduced in the central Agora space, where the teacher presents the task before students begin working. The teacher controls when exercises start, manages transitions between activities, and structures the overall flow of the lesson.
- **Proximity-based voice communication**
  All participants communicate through spatial voice chat integrated into the multiplayer system. Users can hear others who are physically nearby in the virtual space, supporting natural collaboration within groups and between teacher and students.
- **User settings (username and microphone permissions)**
  Each participant sets a username when entering a session. Voice communication requires microphone permissions to be enabled at the headset system level, and correct volume settings are necessary for effective communication.
- **Log in and authorization system (Partner codes and lesson codes)**
  Lesson creation is restricted through a partner code assigned to each school. When a teacher enters this code, a new lesson session is created and a unique lesson code is generated. Students join by entering a shortened version of this lesson code, while teachers use the full code to receive administrative privileges within the session.
- **Teacher role identification**
  The teacher has a distinct visual marker within the environment, making it easy for students to identify who has administrative control and instructional authority during the session.

## High-Level Architecture

This section describes the overall technical structure of the VäXR application and explains how the different architectural components interact at a system level. We hope this provides a clear mental model and overview of how the client application, multiplayer infrastructure, and level structure are organized before diving into implementation details.

### Target platform and build architecture

VäXR is built using Unreal Engine 5.3.2 and is deployed as a standalone Android application (APK + OBB) targeting Meta Quest 2 and Quest 3 headsets. The application is compiled for Android and runs directly on the headset hardware without requiring a connected PC (Standalone).

Although the system is currently optimized for Meta Quest devices, the architecture is based on OpenXR, which makes it possible - though not automatic - to adapt the application to other compatible standalone VR headsets with moderate adjustments.

## Blueprint-based project structure

The project is built almost entirely using Unreal Engine Blueprints rather than C++. This means that the gameplay logic, interaction systems, grouping mechanics, and most runtime behaviors are implemented visually within Unreal's Blueprint system.

The documentation therefore focuses primarily on Blueprint architecture, replication logic, and component relationships, as these form the backbone of the application's structure.

## Multiplayer architecture and server model

VäXR operates as a networked multiplayer system using Unreal Engine's standard multiplayer framework. Each active lesson corresponds to a dedicated Unreal multiplayer server instance running in a Docker container on a virtual private server (VPS).

When a teacher creates a lesson, a backend orchestration system starts a new containerized Unreal server instance. Students and teachers then connect to this instance using a lesson code. Each lesson runs independently, meaning that multiple lessons can run concurrently, each with its own dedicated server process.

Voice communication is handled within the Unreal environment and is not managed by a separate third-party voice service, though it does use a third party voice plugin for audio quality and reliability reasons. This means that both spatial presence and voice replication are integrated within the same multiplayer architecture.

## Session structure and grouping model

Each lesson typically consists of one teacher and up to eight students. Students are divided into groups (A–D), usually working in pairs. The teacher has administrative control over grouping, exercise initiation, and movement between spaces.

The grouping logic is centrally managed within the session, while the exercise logic itself is primarily distributed within the respective interactive tools and environments.

## Persistent level and streamed sublevels

The world structure is based on a persistent main level combined with streamed sublevels. The persistent level contains the central Agora space, session management logic, and shared systems that remain active throughout the lesson.

Exercise rooms are implemented as streamed sublevels. When the teacher sends students out to work in groups, these sublevels are dynamically loaded into the persistent level. Each group operates within its own instantiated room, allowing separation of workspaces while maintaining synchronization through the shared multiplayer session.

This architecture supports both logical separation of exercises and improved performance by ensuring that only necessary content is loaded and active at a given time.

RI.
SE

# 2. Recommended skills

This chapter outlines the minimum skills and competencies required to work on and further develop the VäXR project. It describes the technical and conceptual knowledge a developer should possess in order to effectively understand, maintain, and extend the codebase.

### Skills needed include:

- Unreal Engine and Blueprint proficiency
- Multiplayer architecture understanding
- Backend and container orchestration
- Continuous integration and deployment workflows
- Version control and collaboration
- 3D asset and content workflows

Since the application is built in Unreal Engine and relies heavily on Blueprint-based logic in combination with a custom multiplayer backend, a developer, or team of developers, must be comfortable navigating both interactive engine workflows and structured software development practices. The ability to move confidently between gameplay logic, system architecture, and deployment workflows is essential for maintaining stability and enabling future evolution of the project.

### Unreal Engine and Blueprint proficiency

- Experience with Unreal Engine 5 (UE4 transferable)
- Strong ability to read and modify Blueprint graphs
- Understanding of Unreal Engine architecture
- Basic C++ knowledge (beneficial but not required)

A solid working knowledge of Unreal Engine 5 is essential. Experience with Unreal Engine 4 is transferable, but familiarity with Unreal's architecture in version 5 is strongly recommended. The project is built almost entirely using Blueprints, which means the developer must be able to read, modify, and extend Blueprint graphs with confidence.

While C++ knowledge is not required for everyday development, it is beneficial when working close to engine-level systems, debugging complex behavior, or refactoring performance-critical logic. Because VäXR is a multiplayer application, it is particularly important to understand Unreal's multiplayer replication model, including concepts such as authority, ownership, server-client separation, and replicated variables.

### Multiplayer architecture understanding

- Understanding of Unreal's multiplayer replication system
- Knowledge of authority and ownership concepts
- Ability to structure server-client logic correctly
- Awareness of multiplayer-specific debugging challenges

Since the application runs as a networked multi-user system, the developer must be comfortable reasoning about server-client interactions. This includes understanding how

Unreal handles replication, how authority over objects is assigned, and how gameplay logic must be structured differently in multiplayer contexts compared to single-user environments.

A lack of familiarity with multiplayer architecture can easily lead to unstable or unpredictable behavior, so this competence should be considered fundamental rather than optional.

### Backend and container orchestration

- Basic understanding of Docker containers
- Ability to read and modify simple Python scripts
- Basic familiarity with Node.js / JavaScript
- Understanding of how Parse Server manages lesson data

On the backend side, the system uses containerized Unreal server instances and a lightweight orchestration setup. Basic familiarity with Docker containers, simple Python scripting, and general server concepts is therefore helpful.

The backend interacts with a Parse Server database and uses JavaScript-based server functions, meaning a developer maintaining or extending the backend should be comfortable reading and understanding basic Node.js/JavaScript logic. While deep backend expertise is not required for everyday feature work, the ability to understand how lessons are created and server instances are spawned is important for maintaining system stability.

### Continuous integration and deployment workflows

- Familiarity with GitHub Actions or GitLab CI/CD
- Understanding automated backend build processes
- Awareness of version synchronization between client and server

Continuous integration and deployment workflows are used to automatically build the multiplayer server component when changes are pushed to production branches. Familiarity with systems such as GitHub Actions or GitLab CI/CD pipelines is advantageous, particularly when maintaining or migrating the backend infrastructure.

Understanding how these automated build processes work ensures that new application versions remain synchronized with the correct backend server version.

### Version control and collaboration

- Solid Git workflow knowledge
- Experience with branching and merging strategies
- Ability to handle Git submodules

Maintaining clean commit history and collaboration practices

The project is managed through Git, including the use of submodules for third-party plugins such as the VR Expansion Plugin. A developer should therefore be comfortable with Git workflows, branching strategies, resolving conflicts, and maintaining a clean commit history.

Since multiple developers may work on the system over time, the ability to collaborate using structured version control practices is essential to avoid introducing instability into either the gameplay code or the backend infrastructure.

### 3D asset and content workflows

- Basic 3D modeling workflow knowledge
- Experience importing assets into Unreal
- Understanding materials and material instances
- Knowledge of collision setup
- Awareness of VR performance considerations

For developers extending the learning environment itself, a foundational understanding of 3D asset workflows is necessary. This includes importing models, working with materials and material instances, setting up collisions, and managing assets inside Unreal.

Familiarity with external 3D tools such as Blender or similar software is beneficial when creating or optimizing new exercise components. Even when not directly modelling assets, a developer should understand how geometry, materials, and performance considerations interact within a standalone VR environment.

# 3. Development environment and tooling

This chapter describes the technical environment required to develop and maintain the project. Including the technical environment required to develop, build, and maintain VäXR. It explains which engine version is used, which plugins the project depends on, how the project is configured for XR and Android builds, and what external tooling is required to successfully compile and deploy both the client application and the associated multiplayer server infrastructure.

## Engine version and core setup

VäXR is developed in Unreal Engine 5.3.2, and the project is tightly coupled to this version due to plugin compatibility and multiplayer server builds. While future upgrades are possible, they must be handled carefully to avoid breaking dependencies, particularly those related to third-party plugins and backend container builds.

The project uses an OpenXR-based configuration, allowing it to target standalone VR headsets without being locked to Meta-specific APIs. This architectural choice increases portability but requires deliberate configuration within Unreal's XR settings.

The Enhanced Input System is used for player input management. See VREInputMappingContext (/VRE/Input) for configuration of keybindings. The required XR and rendering settings are already configured within the project file. Developers setting up the environment should especially verify that plugins and general settings remain intact when upgrading engine versions, as they can easily be lost of overwritten.

## Required plugins and dependencies

The project depends on several critical plugins, most notably the VR Expansion Plugin, which provides core VR interaction functionality such as teleportation, gripping mechanics, and multiplayer-aware object authority handling. This plugin is included as a Git submodule, meaning it must be initialized and updated correctly when cloning the repository.

It is important to note that local modifications have been made inside the VR Expansion Plugin codebase—particularly related to remote grabbing and object authority logic (see relevant chapters for more information). When upgrading either the engine version or the plugin version, these modifications may be overwritten. Developers must therefore carefully review changes before performing upgrades.

In addition, the project uses the Universal Cross-Platform Pro Voice Chat Plugin for multiplayer voice communication. This plugin integrates directly into Unreal's networking model and does not rely on an external voice infrastructure.

## Android build configuration and SDK setup

The application is built as an Android APK with an associated OBB file. Proper Android SDK and NDK configuration is required in Unreal Engine to successfully compile the project.

The Android SDK configuration is global across Unreal installations on the same machine. This means that installing or modifying another Unreal Engine version may unintentionally affect the SDK setup used by this project. Developers should verify SDK paths and version compatibility if build errors occur or if the application fails to launch correctly on the headset.

Platform-specific settings for rendering, packaging, and XR configuration are stored within the project settings and should not be altered without understanding their impact on performance and compatibility.

## Version control and submodules

The VäXR project is maintained using Git, and proper Git workflow is essential for stable development. Because the VR Expansion Plugin is included as a Git submodule, developers must ensure that submodules are correctly initialized and updated when cloning the repository.

Care must be taken when merging branches that modify plugin-related logic or multiplayer functionality, as conflicts in these areas can introduce subtle and difficult-to-diagnose runtime issues.

## Backend build and deployment tooling

In addition to the Unreal client application, the multiplayer backend server is built and deployed using a containerized workflow. When changes are pushed to the production branch, an automated pipeline builds a new Docker image of the Unreal multiplayer server. This process typically takes 15–20 minutes.

It is critical that the client application version and the backend server version remain synchronized. Before publishing a new client build, developers must ensure that the corresponding backend container has been successfully rebuilt and deployed. Currently, no automatic version mismatch detection is implemented, meaning inconsistencies may result in unpredictable runtime behavior rather than explicit errors.

## Development workflow considerations

Developing VäXR requires coordination between local Unreal development, version control practices, and backend deployment processes. Changes to gameplay logic, multiplayer handling, or interaction systems may have implications for server builds and replication behavior.

For this reason, developers should adopt a disciplined workflow that includes testing multiplayer sessions after significant changes, verifying container builds, and ensuring that publishing steps align with backend readiness.

**Quick reference overview**

- Unreal Engine 5.3.2 (current project version)
- OpenXR-based setup (portable across XR platforms with adaptation)

- Enhanced Input System (standard UE5 input handling)
- VR Expansion Plugin used (Git submodule). Custom modifications inside plugin (grabbing + authority logic). Submodules must be initialized correctly through version control system (e.g Git, Sublime). Read more and download at: https://vreue4.com/
- Universal Cross-Platform Pro Voice Chat Plugin (needs to be activated in the Plugins menu in Unreal.)
- Android build target (APK + OBB) Android SDK + NDK needs to be correctly configured in Unreal. Remember that SDK configuration is global across UE installations. Instructions can be found at: https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-set-up-android-sdk-and-ndk-for-your-unreal-engine-development-environment?application_version=5.3
- Push to production branch triggers backend build (~15–20 min) No automatic client/server version mismatch detection

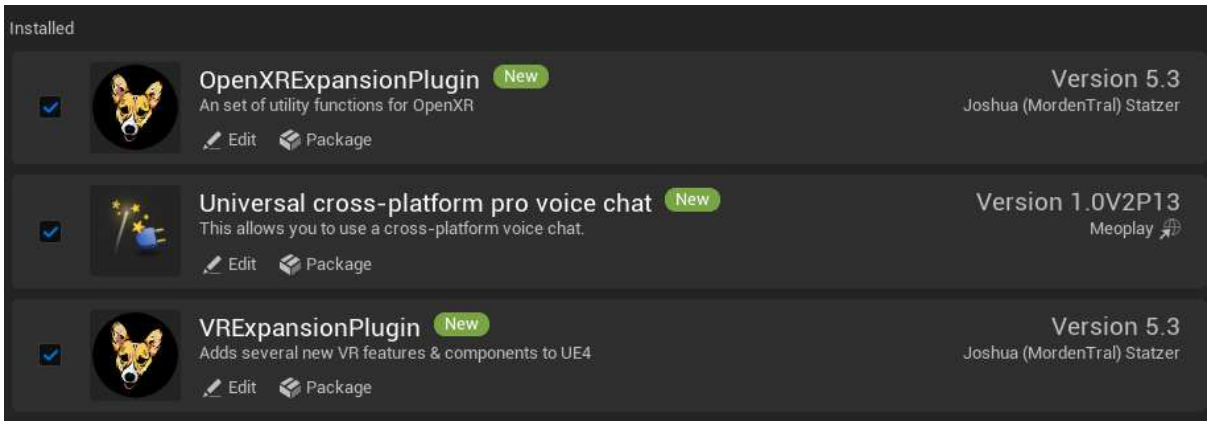



*Figure 1. List of active plugins in Unreal Engine*

# 4. Description of back-end systems

This chapter explains the systems that support multiuser functionality and session handling. It describes how multiuser communication works, the role and setup of the server, and how lessons or sessions are managed, including the use of session codes and structures for different schools and teaching contexts.

The backend architecture connects the Unreal client application running on the headset with a containerized multiplayer server environment and a lightweight orchestration layer. Together, these systems allow teachers to create structured lesson sessions that students can securely join.

## Multiuser architecture – how it works

VäXR uses Unreal Engine's standard multiplayer framework. When users connect to a lesson, they are not simply sharing a peer-to-peer session; instead, they connect to a dedicated Unreal multiplayer server instance. This server maintains authoritative state over all replicated actors, including player positions, interactions, grouping logic, and exercise objects.

Each lesson corresponds to one dedicated server instance. All players connected to the same lesson interact within the same synchronized multiplayer session. Object ownership, authority transfer, and state replication are handled through Unreal's multiplayer model, while the backend ensures that each lesson has its own isolated server environment.

Voice communication is integrated into the Unreal multiplayer system and runs within the same session, albeit with the use of a plugin due to issues with Unreal's default built-in voice solution. This means spatial presence and voice replication are coordinated within the same architectural layer. This voice communication is not handled by an external third-party service.

## Server function, architecture and setup

The multiplayer servers run as Dockerized Unreal dedicated server instances, built through GitHub Actions, hosted on a virtual private server (VPS). Each time a teacher creates a new lesson, a new containerized Unreal server instance is started in the backend.

The orchestration flow works as follows:

1. A teacher enters a valid partner code inside the VR application.
2. The Unreal client sends an authenticated request to the backend (Parse Server).
3. A new lesson entry is created in the database, including a generated lesson code.
4. A Python script continuously monitors the database (currently polling every few seconds).
5. When a new lesson entry is detected, the script starts a corresponding Docker container running the Unreal multiplayer server.
6. Connection details are returned to the client, allowing players to join the newly created session.

Each running lesson has its own dedicated server container, meaning multiple lessons can operate concurrently and independently. These server containers do not require graphical hardware and run purely as headless multiplayer instances.

When a lesson expires (currently based on a time threshold, e.g., after several hours), the database entry is removed through a scheduled job on the hosting platform calling an endpoint on the Parse Server. The monitoring script then detects the absence of the lesson code and shuts down the corresponding Docker container, freeing backend resources.

Technically, this code runs on the Parse Server as a set of Cloud Code functions exposing endpoints. They are written in javascript and located in a main.js file mounted into the Parse server container in a dedicated cloude-code-vol volume.

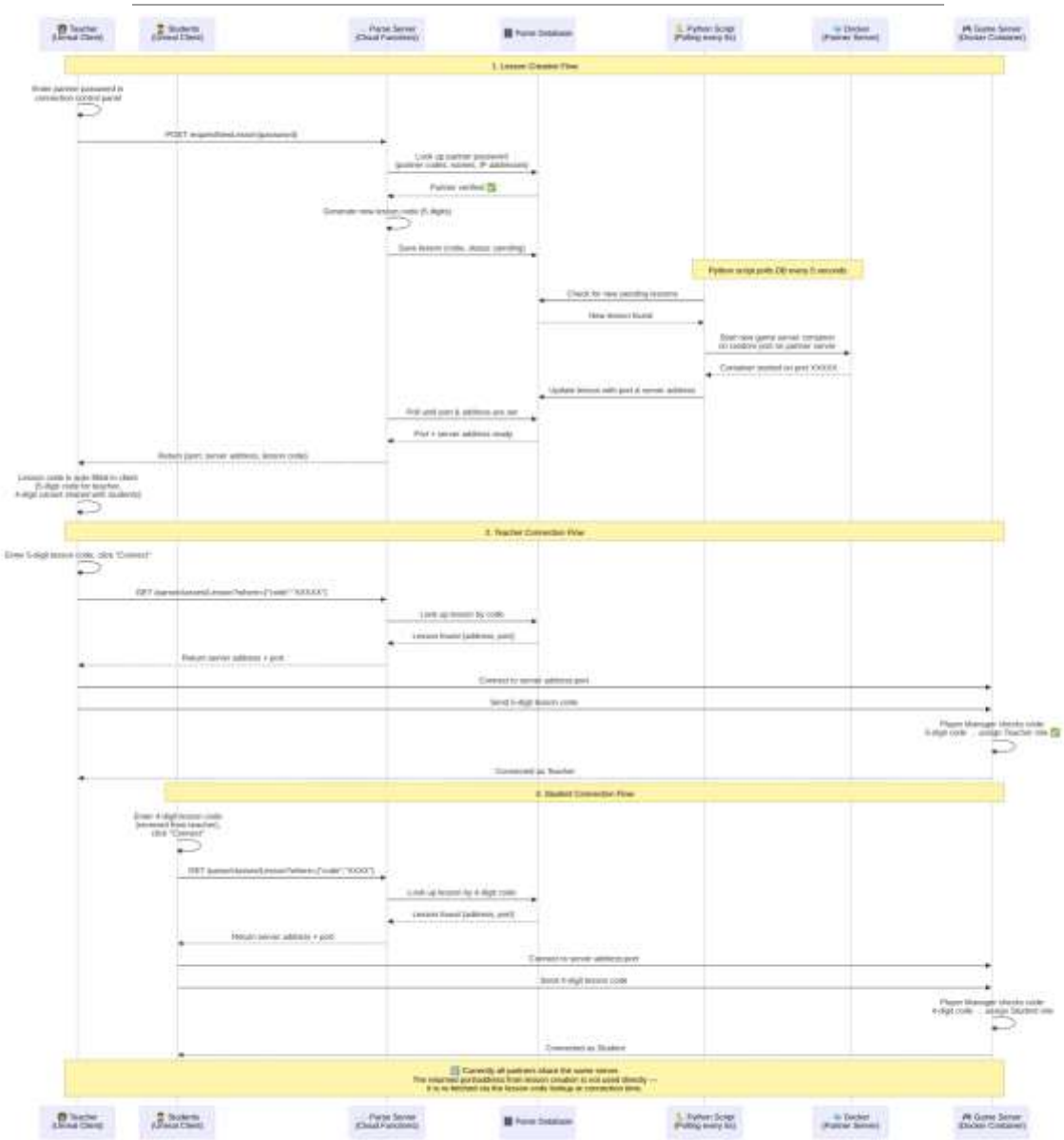


*Figure 2. Lesson creation flow*

## Session ("lektion") management and code system

Lesson access and session creation are controlled through a structured code system.

Each participating school or partner is assigned a partner code. This code grants permission to create new lessons and is stored in the backend database. Partner codes are managed manually through a web-based control dashboard connected to the Parse Server database.

When a teacher enters a valid partner code inside the VR application, a new lesson is created and assigned a five-digit lesson code. This five-digit code serves two purposes:

- The full five-digit version grants teacher-level administrative rights when joining the session.
- The last four digits are shared with students, who use them to join as regular participants without administrative privileges.

From the user perspective, there are effectively two code types: the partner code (for creating lessons) and the lesson code (for joining lessons). Internally, authentication tokens are used to secure communication between the Unreal client and the backend server endpoints.

Students enter their four-digit lesson code in the VR interface. The client sends a request to the backend, which verifies that the lesson exists and returns the connection details for the appropriate Unreal multiplayer server instance. The headset then joins the correct multiplayer session as a client.

## Administrative interface and database management

The backend includes a web-based control panel connected to the Parse Server database. This dashboard allows administrators to:

- Add or manage partner codes
- View active lessons
- Inspect database entries

Currently, adding a new school requires manually inserting a new partner entry in the database through this interface.

The backend orchestration is hosted and managed through a server control system (e.g., Coolify), though this is an infrastructure choice rather than a hard dependency of the codebase.

### Quick reference overview – backend

- **Unreal dedicated multiplayer server per lesson**: One Docker container spun up for each active lesson. Hosted on VPS (headless server, no GPU required)

- **Lesson creation flow:** Teacher enters partner code in VR. If the code is valid the backend starts a new containerized lesson server and returns a 5 digit lesson code. The

teacher can use this code to connect to the lesson server and get special teacher privileges, and give a shorter 4 digit version of it to their students to allow them to connect.

- **Code system structure:** Partner code = permission to create lessons (per school). 5-digit lesson code = teacher access (admin role) 4-digit lesson code = student access (client role)
- **Database and orchestration:** Parse Server manages partner codes + active lessons. Python script polls database and manages container lifecycle. Expired lessons automatically shut down (time-based cleanup)
- **Voice communication:** Integrated into Unreal multiplayer session. No external voice service. On Meta Quest headsets mic permissions for the VäXR app are crucial for voice communication to work.
- **Administrative tools:** Web-based dashboard for managing partner codes. Manual addition of new schools in database. Backend hosted via server control platform (e.g., Coolify).
- **Important architectural notes:** there is no automatic client/server version validation so strange bugs may occur if clients are not up to date. Short numeric codes means there is a limited amount of security if a dedicated attacker wanted to gain access to lesson instances. Backend must be rebuilt when production branch is updated. Multiple lessons can run concurrently (isolated containers)

For more information on current limitations and architectural considerations see chapter "Known Issues and Technical Debt".

# 5. Description of real-time systems

This chapter focuses on the interactive systems that operate during runtime, meaning the capabilities available to the user as they step into the application. It describes how core interaction mechanisms such as teleportation, object interaction, distance-based interaction, and teacher tools are implemented and how they function together to support real-time use of the application. This differs from Lessons which in contrast cover interactive learning objects in the environment. This chapter instead focuses on the immediate abilities of the user's avatar, and its supporting systems.

**This chapter covers**

- VR player controller (player avatar)
- Teleportation (NavMesh)
- General interaction (grabbing and throwing)
- Laser pointer and grabbing objects from afar
- Teacher panel and tools

## VR player controller (player avatar)

This part covers the details of implementation of player control logic. Close to all interaction logic is supplied by the VRExpansionPlugin. This means that the online documentation and resources around the VRExpansionPlugin is the best and most up to date documentation. If you intend to change or expand on these feature sets, we direct you to their online resources at https://vreue4.com/. In VäXR many of these features are used as is (except for certain grabbing logic, see the section on Laser pointer in this chapter. This section will nonetheless provide cursory orientation of the VR expansion toolkit and its use in the project.

Please keep in mind that the VR Expansion toolkit is an ongoing project where code is updated continuously by other developers. This means that functions may change and update, which may improve core interaction functionality for the player, but may also disrupt blueprint functionality in the VäXR project. It is recommended to keep the VR Expansion plugin at the recommended version for the particular version of Unreal Engine used.

### Template_VR_Player_Controller (/VRE/Core-)

This sets up the VR pawn and does general housekeeping for the player, such as initializing voice chat. It also relays client-side RPC commands to the server-only game mode. See VR Expansion Toolkit for more information.

### BP_VRCharacter (/VRE/Core/Character)

This actor/pawn is a subclass of the *VRCharacter* class from the VRExpansionPlugin. It handles various general player interaction logic.

The model of the player is made up of a few basic shapes. These are modified depending on the state of the replicated Teacher boolean, through the On Rep B Teacher function. A similar modification of the nameplate above the player's head happens through the On Rep Player Name function.

This class currently handles a couple of RPC calls from UI code to the server, SetPlayerNameOnServer and InformSserverOfLessonCode. To be more consistent with other UI code (see Details of Implementation of Centralized Logic and UI) these could be moved to the player controller instead, Template_VR_Player_Controller.

## Teleportation (NavMesh)

### Teleportation in Unreal Engine and the VR Expansion Plugin

In Unreal Engine, teleportation in VR is built on top of the engine's navigation system, meaning that teleportation is constrained by where the engine considers movement to be valid. Instead of allowing teleportation to any surface hit by a trace, Unreal validates potential landing locations against the navigation mesh (*NavMesh*). This ensures that players can only teleport to walkable, approved areas.

The VR Expansion Plugin relies and builds on top of Unreal's navigation system. Teleportation works by first calculating a potential destination—typically using a parabolic arc trace—and then validating that location against the NavMesh. If the location is considered valid and walkable, the player is moved there. If not, teleportation is rejected.

In practice, this means that teleportation is navigation-driven movement rather than free relocation. The NavMesh defines the legal movement surface, and the VR Expansion Plugin uses that data to safely reposition the player.

The user activates teleportation by tilting any joystick forward. As the joystick position is released the teleportation action excecutes.

### How the VR Expansion Plugin handles teleportation

- Uses a projectile-style arc trace to determine a potential landing location.
- Validates the landing point against Unreal's NavMesh to ensure it is walkable.
- Checks constraints to avoid invalid surfaces.
- Rejects teleport attempts that fall outside the navigation mesh parameters.
- Relocates the VR Pawn only after the location has been validated by the navigation system.

For teleportation to function correctly, the level must contain a valid NavMesh, generated within one or more NavMeshBoundsVolumes, and the walkable geometry must have proper collision.

### NavMeshBoundsVolume and how it works

Teleportation requires a *NavMeshBoundsVolume* placed in the level.

A NavMeshBoundsVolume defines a 3D region in which Unreal is allowed to generate navigation data. Within this volume:

- The engine scans collision geometry.

- Walkable surfaces are identified.
- A navigation mesh is generated.

You can use multiple NavMeshBoundsVolumes in a level. They can be resized to tightly wrap specific rooms, placed in combination to cover complex layouts.

By piecing together several volumes, you can control exactly where teleportation is allowed. If a surface is not inside any NavMeshBoundsVolume, it will not become part of the NavMesh, and teleportation will fail there. Adjusting the height of the volume is important to avoid having students climb on top of wall segments and escape the confined exercise spaces.

### Recast NavMesh (the underlying system)

Unreal Engine uses Recast NavMesh as its default navigation system. This is the system that analyzing level geometry, determines walkable areas based on slope, step height, and agent size. And generating the navigation mesh overlay visible in the editor (when pressing "P").

The NavMesh must be built (either automatically or manually) before teleportation works correctly. See official Unreal Engine documentation for more precise considerations and up to date instructions.

## General interaction (grabbing and throwing)

The VR Expansion Plugin implements grab–drop interaction through a structured gripping system rather than simple attachment. Instead of directly attaching objects to a motion controller, the plugin uses a dedicated Grip system that manages ownership, physics behavior, and replication in a consistent way.

The grip system supports multiple grip types, physics interactions, and configurable replication settings. It is more advanced than simple attach-to-component solutions and is specifically built to function reliably in multiplayer VR contexts.

When a player attempts to grab an object, the controller checks whether the object implements the required grip interface. If valid, the plugin establishes a grip relationship between the motion controller and the object. This relationship then determines how the object behaves.

Dropping an object reverses this relationship. The grip is released, and the object either re-enables physics simulation or returns to a predefined state depending on how it was configured.

### Multiplayer behavior and authority – Standard Grab-Drop Interaction

In multiplayer sessions, the most important aspect of grabbing is authority management. Unreal's multiplayer model is server-authoritative, meaning the server ultimately controls object state. The VR Expansion Plugin handles this in the following way:

- The client requests grip authority.
- The server validates the request.

- Authority over the object is assigned appropriately.
- Object movement is replicated from the authoritative side to all other clients.

This prevents two players from fully controlling the same object at the same time and keeps object state synchronized across the session.

Standard grab-drop interaction for the player is triggered through triggering the grip button (squeezing the middle finger around the Quest controller for example). It is necessary for the virtual hand of the user to collide clearly with a grabbable object for grabbing to take effect.

For the purpose of this document, we will refer to this functionality as *Standard grab-drop interaction*.

### Laser pointer and grabbing objects from afar

The VR Expansion Plugin also has an implemented laser pointer. This can be activated (toggled) on each hand.

Controls (on a Meta Quest 3 controller):

- Right hand - B button
- Left hand - Y button.

Once activated a red line appears as well as an underlying casted ray for querying hit objects in the scene.

In VäXR this laser line can be used to visually point towards objects, but it is also compatible with grabbing designated objects from afar. To grab object from afar the user needs to have an active laser pointer and squeeze the grip button on the same controller.

An object is then transported to the users hand using a modified version of *VR Expansion Plugins Toss function*. The blueprint nodes are called *VäXR gripping code* triggering the event *ServersideTeleport_EGEN* and is implemented into the *physicsTossManager* in *BP_VRCharacter*.

This code is currently only implemented to grab 3D objects in exercise 2 but can be activated for use to grab any object with a collider.

WARNING: This is a direct modification of the code inside the VR expansion Toolit system. These changes will be overwritten if, for example, a new version of the toolkit is installed.

## Control panels and teacher tools

The current UI elements consist of a control panel (*A*_ConnectMenu) placed near the player when they enter the VäXR experience, and a teacher menu (*A_TeacherMenu*) only visible to the teacher, attached to the arm of the teacher pawn for easy access in VR.

### Connection Menu (/UI/A_ConnectMenu)

The appearance and functionality of this varies depending on if they are offline or have joined a multi user session. It allows them to set a name for themselves and enter the codes necessary to start and/or join lessons (see Session management and code system).

### Teacher Menu (/UI/A_TeacherMenu)

This is only visible for the teacher, and allows them to manage students and the flow of the lesson. They can assign the students to groups, send them out to exercise rooms, gather the class, and travel together out to exercise rooms. Additionally the teacher can temporarily disable player movement if deemed necessary.

### UI implementation

UI is rendered as widgets on actors placed out in the world (or as is the case with the teacher's control panel; attached to a pawn). These widgets contain their own logic, but will do RPC calls on server-synchronized classes like *Template_VR_Player_Controller* and *BP_VRCharacter*. For more info see Level Structure & World Logic.

# 6. Description of exercises

This chapter documents the educational content of the application. It explains how the different geometry lessons are structured and implemented, where their logic and Blueprints can be found, and how they integrate with the rest of the system.

There are three exercises ("lektionsverktyg") implemented in VäXR.

1. Allows building 3D shapes out of 2D shapes,
2. Allows sorting 3D shapes into frames,
3. Allow students to explore volume by filling 3D shapes with water.

In this chapter we break down each; what it does, how it works, where it is located in the project, and is connected with other systems.

Note: All BPs have been sorted per exercise meaning all BPs needed are found in their respective folders. The only exception is the BP_ButtonToggle which is used as a toggle button across exercises. For that reason it is placed directly in the Blueprints-folder.

Note: Folders containing BPs are named Exercise1 -3 respectively. These contain the interactive systems (BP:s) that make up each exercise. This contrasts with the Rooms which are containers and instances that contain the exercises.

## Toggle Button

### What it is

The toggle button is an interactable button, placed on a pedestal with an optional descriptive text. It can be found in Exercise 1 next to each pedestal where shapes are attached, and in Exercise 3 near the water taps. The blueprint *BP_ButtonToggle* is located in /Blueprints.

### What it is for

In Exercise 1 where it controls folding of the shapes, and in Exercise 3 two buttons are used to fill and empty the shapes respectively.

### How it works

Events can be attached to the buttons event dispatchers. These are called when the hand of a user overlaps with the button. The event dispatchers are executed on all clients simultaneously through reliable replicated RPC calls through the server.

## Exercise 1 - Build 3D figures out of 2D sides

### What it is, and where it is

All BPs related to this Exercise (except *BP_ButtonToggle)* are located in /Blueprints/ Exercise_1. During the majority of development it was intended as the second exercise, but this changed after playtest feedback.

Exercise 1 BPs consist of:

- **BP_2D_V**x* The 2D shape actor that can embody any loaded 2D shape.
- **BP_Pedistal _V**x* The pedestals that the BP2D is placed upon to start building)
- **BP_foldPoint_V**x* The FoldPoint anchor any other 2D shape and allows that shape to be folded according to an IK joint.
- **BP_Lever** Allows the user to grab a handle and manually manipulate the 2D shape fold angle.
- ***BP_2D_ShapeSpawner*** Spawns new 2D shapes after a timelimit.

* The x stands for a version number. The blueprint folder may contain multiple versions of the same BPs. Higher number denotes a later version.

Any additional BPs in the folder can be considered support functions and are either children of any for the above blueprints or data containers (such as Enums) or 3D sound assets played when a 2D shape is attached to another.

## What it is for

Exercise 1 (sv: Övning 1) allows the students to collaboratively, or alone, explore the components of a three-dimensional shapes by constructing them using 2D shapes. Several variants of 2D shapes hang on a wall (squares, rectangles, triangles, and hexagons) in several instances.

**Sequence:**

- When spawning a 2D shape is determined, and a fold point per side is spawned as a child of the 2D actor
- A foundation can be made by grabbing a 2D shape and moving it over to a Pedestal
- Letting go of the shape within a certain radius of the top of the Pedestal top attaches the 2D shape to the pedestal as a base.
- The shape now activates its sides (*FoldPoints*), and any additional 3D shape can have its side (FoldPoint) attached to the placed (rooted 2D shape)
- Grabbing any 2D shape "activates" its sides making them possible to attach to active sides on any other 2D shape who also have active sides.
- If the sides of two active 2D shapes are held close to each other, they enter "pairing mode", which allows the two connectable sides to attach to each other if they are let go. All other fold nodes become inactive, stopping them from accidentally attaching to any other active FoldPoints nearby.

## How it works

***BP_2D_Vx***

- Key interaction – BP_2D is an actor extended from one of the objects included in the VR expansion toolkit. This makes it a grabbable object with functionality ready for multi-use interaction, which is a key feature set of the VR expansion toolkit. This handles replication, authority, and syncing over multiplayer sessions, for the grabbed object. *Standard grab-drop interaction* applies. This object cannot be launched or grabbed from afar (in the way exercise 2 – objects can).

***BP_Pedistal***

- The pedestal acts as a base plate for the first 2D shape. After a 2D shape has been attached to the pedestal it updates the 2D shape to list as attached to the base. This is tracked and updated for each instance of a 2D shape though the AttachStates Enumerator.
- The AttachState determines the possibility for other 2D shapes to attach to it. To be attachable (ReadyToAttach in the AttachStates Enum) a 2D shape must be attached to the pedestal or to a 2D shape which is itself registered as attached in the AttachStates Enumerator (2D attached).
- The pedestal is also responsible for tracking the attached children with its array of 2DMinions.
- An Event dispatcher handles connection between the BP_ToggleButton and update all attached 2D shapes to trigger their fold functions through the node FoldUpd2Dshape andFoldDown2DShape.

***BP_FoldPoint***

- The main role of the FoldPoint is to act as an attachable and foldable child on any other object it is itself a child of.
- FoldPoints are attached as a child to the Pedestal (spawned at BeginPlay) and on each side of the 2D objects (spawned dynamically when grabbed
- They handle collision and attach check logic for all objects colliding with it and determine if it is another 2D shape and that it is in an attachable state.
- If attachable, it takes the parent object of the collided FoldPoint (BP_2D) and, through the Attach Component to Component – node, attaches it as a child to the itself.
- When triggered by an assigned BP_ToggelButton the FoldPoint also is responsible for folding the child object (meaning another 2D shape and any additional 2D shapes attached in the hierarchical chain) to the predetermined angle.

***BP_Lever***

- The lever is a child object spawned by the BP FoldPoint when players intersect the parent 2D Shape.
- It allows users to see and grab hold of a knob (child actor) and pivot the parent 2D shape to any desired angle within its axis of orientation and degrees of constraint.
- The BP_lever is technically a spawned lever object form the VR Interaction Toolkit which is added upon interaction and then removed after a time limit.

***BP_2D_ShapeSpawner***

- Simply spawns new BR_2D shapes after a time limit, if the Spawner is not already blocked by a 2D shape.

Beware that this logic may come to evolve over time. Please make sure you have the most up to date documentation.

## Exercise 2 – Sort shapes

## What it is, and where it is

All BPs related to this Exercise are located in **/**Content Folder/ Blueprints /Exercise_2.

Exercise 2 BPs consist of:

- **BP_3D_shapes** * The x stands for a version number. The blueprint folder may contain multiple versions, where later = better.
- **BP_gridMaker_01**
- **BP_SortingBoardx***

* The x stands for a version number. The blueprint folder may contain multiple versions, where later better.

## What it is for

Exercise 2 (sv: Övning 2) allows the students to sort 3D shapes (BP_3D_shapes) into frames (SortingBoards).

**Sequence:**

- A student can pick up a 3D shape manually (*Standard grab-drop interaction*), or from afar using the laser pointer.
- While holding the 3D shape the student can throw/ launch the shape forward by pressing the trigger button.
- If the shape hits a frame (SortingBoard) the Sorting Board decrease the size of the 3D shape and attaches it to itself
- The student can grab the shape again or launch a new one on the same board.
- Any additional shapes launched at the board will sorted according to an array function thus stacking them up neatly inside the frame (SortingBoard).

## How it works

**BP_3D_shape**

- The BP_3D_shapes are placed at design-time and configured to appear as any of 25 different 3D shape.
- The function can be configured in Shape Selection and any alternative mesh added selected in design time though a public variable under the objects Default in the editor.
- These 3D objects mostly have functions and nodes related to adapting to what happens to them as they are grabbed and attached. They contain simple animations and states triggered by the Sorting boards, player interaction.

**BP_gridMaker_01**

- The *GridMaker* creates a reference grid for the *Sorting Boards* to spawn upon.
- It allows defining size and number of SortingBoards Spawned using the public variables *Rows* and *Columns*, which then generate and save *SortingBoard*s in the array Boards.
- Boards are spawned

- The Grid maker is also responsible for updating the state stored for each board in the Enumerator *E_SortBoardAnimStates*. This is done though the function *CycleAll Boards*. This is so that boards can be both spawned and hidden (made inactive) when any of the board is updated. This allows the whole grid to expand and contract as the user adds or removes shapes from the SortingBoards across the grid.

**BP_SortingBoard**

- The *SortingBoard* handles collision logic and validation as it is hit by a *3D_shapes* actor.
- When hit It attaches the *3D_shapes* actor and updates its state (*PlaceShape*) and keeps track of the 3D shapes though the array *ActorsHeld*.
- It also places the 3D shape into an available slot on the board and updates the GridMaker
- The *OnRep_BoardState* function sets board state depednign on situation and the On Rep configuration keeps clients and servers up to date with its status.
- As the Sorting board detects the collision of an 3D Shape (using On Component Begin Overlap and On Component End Overlap -events) it also updates the parent GridMaker trough the CycleAllBoards function.

## Exercise 3 - Fill shapes

### What it is, and where it is

All BPs related to this Exercise are located in /Blueprints/Exercise_3/. This includes the m_Fill material which is animated to create the illusion of water filling the 3D shape.

Additional components such as the Fill Shape 3D which are located under /3D/FillsShapes/. The BP_Faucet is located under Content/3D/Faucet. The reservoirs seen over the cranes are simply a *Fill_Cubiod* 3D object.

Exercise 3 BPs consist of:

- **BP_FillShapes**
- **BP_PlacementPlate**
- **BP_Faucet**

### What it is for

Exercise 3 (sv: Övning 3) allows the students to sort 3D shapes (*BP_FillShapes*) by placing them on plates (*BP_PlacementPlate*). Each PlacementPlate is located under a big tap (BP_Faucet) with reservoirs of water.

**Sequence:**

- A student can pick up any of six 3D FillShapes form the corner of the room. The *FillShapes* are configured to appear as unique 3D shapes (Pyramid, cylinder, cuboid and so on)

- The students place these instances in a row under the taps (faucets) in the room on top of a *PlacementPlate*.
- As the assigned *BP_ToggleButton* is pressed the faucets opens up and fills the corresponding 3D shape with the exact amount of water containable by that shape.
- This allows the students to see how much water any object can contain compared to any other 3D shape, and how much water is left in the reservoir afterwards.

## How it works

**BP_FillShapes**

- Much like the other grabbable 3D objects the *FillShape* are instances of the same BP but is configurable to take the shape of, and behave, as any of the selectable 3D objects. This is set up and selected in the function *F_ShapeSelection*.
- Each *FillShape* is using the material m_Fill which is animated full and empty when triggered by the associated BP_ToggelButton.
- Upon BeginPlay the FillShape attached to EmptyOfWater and FillWithWater though event dispatchers.
- A fill shape is attached to a *PlacementPlate* socket after having validated it as non-occupied.
- When the BP_ButtonToggle is pressed the EmptyOfWater and FillWithWater events are triggered according to set up event dispatchers.
- Each FillShape animates the variable FillValue though a timeline which has separate animation tracks for the different 3D shapes to simulate the water level rising.
- Sets the new *Scalar Parameter Value* of the associated DynamicFillMaterial associated with the 3D shape.

**BP_PlacementPlate**

- The placement plate is mostly a reference object without logic. It has a boolean (*Occupied*) to check its availability and prevent stacking. All other logic is updated through the other blueprints.
- Note that the PlacementPlate 3D model used is configured with a socket which is referenced under attach actor to component in the Fill Shapes event graph (Attach Actor to Component node).

**BP_Faucet**

- The *Faucet*, like the PlacementPlate is also a simple blueprint with little logic. It is responsible for some animations such as open/close.
- It has a column of water (WaterCylinder) which animates according to a *Timeline* when triggered. It toggles to fire and execute in response to events sent from the BP_ToggleButton though its functions OpenFaucet and CloseFaucet.

# 7. Level Structure & World Logic

This chapter describes how the virtual environment in VäXR is structured and how gameplay logic is distributed across levels and actors. The system is intentionally designed to separate persistent session management from exercise-specific environments, allowing the application to support structured lessons while maintaining flexibility and performance.

## Persistent level as gathering spot

At the core of the application is a persistent main level, *LabPersistent*, which remains loaded for the entire duration of a lesson. This level contains the Agora (central gathering space), with examples of the various exercises. The game mode, *VRGameMode*, is attached to this level and is therefore persistent throughout the use of the app.

## Streamed sublevels as exercise rooms

Exercise rooms are implemented as streamed sublevels that are dynamically loaded by name (see Room Manager) into the persistent level. When the teacher initiates an exercise and sends students to work in their groups, the corresponding sublevels are created and loaded.

Each group (typically A–D) operates within its own instantiated room. These rooms are recreated when exercises are restarted, which effectively resets the learning environment. This design allows multiple groups to work simultaneously in separate spaces while still remaining part of the same synchronized multiplayer session.

The use of streamed sublevels serves both organizational and performance purposes. It prevents unnecessary actors and tools from being active at all times and allows rooms to be reset efficiently by destroying and respawning the sublevel.

Each exercise room sublevel can be found in /Sublevels/ and is named after the exercise number and the group number, such as Room1_B for exercise 1 for group B. Some customizing is done to each exercise room for each individual group, such as tagging certain actors with the group name. However, all shared geometry and actors for exercise rooms exists as a sort of base levels in the root of the project's content folder. These are named Exercise 1, Exercise 2 and Exercise 3. These are referenced as Level Instances in each of the exercise room sublevels, and any changes to these base levels will automatically be reflected in the group-specific sublevels.

## Distributed exercise logic

VäXR distributes most exercise logic directly into blueprints attached to actors within the rooms. The pedagogical tools themselves contain their own interaction logic, state handling, and behavior.

This means that the learning mechanics are not tightly bound to any level structure. In most cases, tools could theoretically be placed anywhere in the environment and still function correctly, as their logic is self-contained.

An exception exists in the first exercise, where certain interaction dependencies make it less modular than the others. This exercise is currently more tightly integrated with the persistent level due to reset and attachment behavior, which limits its flexibility compared to later exercises. To ensure that exercise 1 functions properly in current state, it should be placed and maintained in the persistent level.

## Centralized grouping and control logic

While exercise mechanics are distributed, certain systems remain centralized.

The functionality of setting teacher role, assigning players to groups, teleporting them, and locking their movement is implemented as custom events in the Player Manager actor component (*AC_PlayerManager*) attached to the *VRGameMode*.

## Room lifecycle and synchronization

Analogous to the Player Manager, the Room Manager (*AC_RoomManager)* handles the logic for creation and destruction of exercise rooms and is attached to the VRGameMode.

Room creation and destruction are synchronized across all connected players through Unreal's multiplayer replication system. When a room is spawned or removed, all clients receive the updated state through server authority.

When the teacher sends students to their exercise rooms, the system destroys existing rooms and recreates them. This reset process ensures that all groups start from a clean state. However, it also means that resetting one group's environment currently affects all groups simultaneously.

Occasionally, synchronization issues may occur where a room is not correctly instantiated for all participants. In such cases, restarting the exercise (which triggers room recreation) may resolve the inconsistency.

Keep in mind that resets may not take effect on exercise 1, if instances are maintained in the persistent level.

## Architectural flexibility

As there currently is little central logic determining how exercises operate, new ones can relatively easily be added by adding in more exercise rooms through creating sublevels, and updating the room management GUI and the Room Manager component.

The fact that the pedagogical tools are relatively self-contained also allows a greater flexibility in adjusting the exercises and reusing the tools in new exercises altogether.

### Quick reference overview

- **Persistent main level (always loaded)** which contains the Agora. Handles session logic, grouping, spawning and overall lesson control. Acts as the structural backbone of the session.
- **Streamed sublevels (exercise rooms)** dynamically loaded when the teacher starts an exercise. One instantiated room per group (A–D). Destroyed and recreated on exercise restart for reset purposes. Used for structural separation and performance optimization.
- **Distributed exercise logic** primarily implemented inside actor Blueprints attached to tools within each room. Exercises are largely self-contained and not dependent on separate GameModes. Exercise 1 is less modular and partially tied to the persistent level.
- **Centralized session systems** manage grouping, room spawning, student distribution and teacher-triggered transitions at the session level rather than within individual exercise rooms.
- **Room lifecycle behavior** where restarting an exercise recreates all rooms simultaneously. No per-group reset currently exists. Room state and spawning are synchronized through Unreal's server-authoritative replication model.
- **Architectural design principles** based on separating persistent session logic from exercise logic, enabling modular tool development and scalable expansion while maintaining performance through level streaming.

## Details of Implementation of centralized logic

As is typical in the Unreal Engine multiplayer architecture, the game mode only exists on the server. This functionality is often triggered through widget events on the teacher's control panel. Widgets however are strictly client-side, so the events are first replicated to the server through the player controllers, Template_VR_Player_Controller. From here, the events are then triggered on the game mode and its attached actor components. https://cedric-neukirchen.net/docs/multiplayer-compendium/common-classes/playercontroller/#examples-and-usage

### Player Manager (/Blueprints/PlayerManagement/AC_PlayerManager/)

This component keeps a list of all active groups, i.e. groups with at least one player in them. This is used when creating rooms to ensure only the rooms for the active groups are created.

### Room Manager (/Blueprints/PlayerManagement/AC_RoomManager/)

Loads and unloads the streaming levels used for exercise rooms.

The naming convention for exercise rooms (Room1, Room2, Room3) is defined by the RoomNames enum, and the naming convention for player groups (A, B, C, D) is defined by the PlayerGroupNames enum (both in /Blueprints/PlayerManagement). AC_RoomManager

constructs string arrays of these at startup to keep track of the current state of loaded exercise rooms, and to use for loading streamed levels by name.

The call to create the rooms comes from the Player Manager as part of the event to send players out to rooms, and is then actualized by a reliable multicast RPC on the game state instance (*BP_VRGameState*). This way all clients simultaneously load rooms as streamed levels, with latecomers or reconnecting users loading the levels upon connecting.

### *BP_PlayerPoint*

These actors are placed in the Rooms to denote spots where players can be safely teleported, and tagged with the name of the group that the room belongs to. This is how players get teleported to the right exercise room for their group, and to avoid players landing inside objects or other players.

### *BP_GatheringPoint*

This fills a similar role to the player point, but denotes a spot in the Agora where players can be teleported back when the teacher wishes to gather everyone there. They are spread out slightly around it to avoid collisions.

# 8. Optimization

This chapter describes the current optimization strategy used in VäXR. Because the application runs on standalone VR headsets, performance considerations are central to both technical and design decisions. The system is built to balance stability, clarity of educational content, and flexibility during development. While there is room for refinement, the current structure provides a workable foundation for multiplayer learning sessions.

## LOD system

Level of Detail (LOD) functionality is supported and used where appropriate to reduce geometric complexity at distance. Larger assets and environmental elements benefit most from this approach, helping maintain performance when multiple players share the same session.

The LOD setup is functional but not yet fully standardized across all assets. As the project stabilizes, expanding and systematizing LOD usage will further improve performance without affecting learning interactions.

## Distance occlusion culling and VR constraints

Distance-based occlusion culling is one of the primary performance tools currently in use. Objects beyond defined distance thresholds are culled to reduce rendering cost, which is particularly important in a multiplayer VR environment where multiple rooms and actors may be active simultaneously.

Given the constraints of standalone headsets, maintaining frame stability takes priority over visual complexity. Distance culling has proven to be a practical and predictable solution during development. While more advanced visibility techniques are available in Unreal, the current approach supports both performance and iterative workflow without introducing excessive rebuild times.

## Post processing and anti-aliasing

Post-processing effects are kept intentionally minimal. This and other visual effects in post processing are avoided to preserve frame rate and reduce GPU load.

Anti-aliasing has been tested and adjusted carefully. Stronger settings can improve edge smoothing but may negatively affect text readability and performance. Current configurations prioritize stable interaction and clear visual communication over graphical polish. However currently the project employs TextRenderActors which are flexible to update but not the most readable as letters are negatively affected by aliasing. Anti-aliasing in contrast makes the texts blurrier and harder to read as well. Read more under Known issues and technical debt.

## Rendering settings and lighting strategy

The project currently relies on dynamic lighting rather than baked lightmaps. This decision reflects the evolving nature of the environment, where rooms are frequently reorganized or adjusted. Using dynamic lighting allows the development team to modify spatial layouts without needing to rebake lighting after each change.

A single primary directional light is used to keep the environment consistently lit while limiting the number of active light sources. This setup represents a balanced compromise between flexibility and performance. If the level structure becomes more static in the future, transitioning to baked lighting may offer additional performance gains.

## Known performance drains

Several areas have been identified as performance-sensitive. A high number of small actor-based objects can introduce overhead, especially in a multiplayer context where replication increases cost.

Instance-based environmental elements, such as wall sections, may also contribute to unnecessary rendering overhead if not consolidated properly.

Blueprint-heavy interaction logic can introduce runtime cost if overly complex or poorly structured. While the system functions reliably, further optimization of blueprint architecture would likely improve both clarity and performance.

# 9. Known issues and technical debt

This chapter documents known problems and limitations in the project. It highlights fragile systems, hard-coded logic, temporary solutions, areas that require extra testing when modified, and considerations or proposals for future improvements. VäXR is a working system used in real educational contexts, but it is also an evolving platform. Some architectural decisions were made pragmatically during development, and certain systems would benefit from restructuring, hardening, or refinement in future iterations.

## Backend version synchronization

At present, there is no automatic version validation between the headset application and the backend multiplayer server container. If a new client version is published without rebuilding the corresponding backend container—or if users connect from different release channels—it is possible for clients to connect successfully but encounter unstable or undefined behavior.

This is manageable in a controlled development workflow but would benefit from explicit version checking between client and server to prevent mismatches.

The backend currently relies on relatively short numeric codes for authentication. While sufficient for controlled educational environments, this approach may require strengthening if the system scales significantly.

There is also no automatic version mismatch validation between client and server builds. If the backend container and headset application versions differ, users may connect successfully but encounter unstable or undefined behavior. For this reason, synchronizing client releases with backend builds is critical during deployment.

## Code-based authentication model

Lesson access is currently controlled through short numeric codes. While sufficient for educational use within known partner schools, this model relies on relatively simple authentication logic. As the system scales or expands beyond controlled contexts, the authorization flow may need to be strengthened with more robust identity management or session validation mechanisms.

Additionally, certain authentication tokens are currently embedded in the client application. While protected through standard backend validation, this structure could be made more modular and configurable.

## VR Expansion Plugin modifications

The project includes local modifications inside the VR Expansion Plugin, particularly related to grabbing behavior and authority handling in multiplayer contexts. These changes are functional but tightly coupled to the current engine and plugin version.

Upgrading Unreal Engine or the plugin may overwrite or conflict with these modifications. A more modular approach—where custom logic is layered on top rather than embedded

directly—would reduce upgrade risk.

### Exercise 1 structural fragility

The first exercise includes complex attachment and detachment logic between multiple interactive objects. Because of these dependencies, it is less modular than later exercises and is partially tied to the persistent level rather than being fully self-contained in a streamed sublevel.

In multiplayer contexts, simultaneous interaction with shared objects can occasionally produce unexpected states. While Unreal's authority model and the VR Expansion Plugin mitigate most conflicts, the logic could benefit from simplification and clearer separation of responsibilities.

### Room spawning and reset behavior

Exercise rooms are destroyed and recreated when a lesson is restarted. This provides a clean reset state but currently applies to all groups simultaneously. There is no per-group reset functionality, meaning that resetting one group's exercise resets all groups.

In rare cases, room spawning may fail to synchronize correctly for all players. Restarting the exercise typically resolves this, but the spawning logic could be made more robust.

### Voice communication stability

The voice system has been replaced multiple times during development. While the current implementation has proven sufficiently stable to support lessons, occasional inconsistencies have been observed in session behavior.

Additionally, microphone permissions must be manually enabled at the headset system level. If permissions are reset during application updates, users may appear muted without any in-app notification. A clearer permission-check mechanism within the application would reduce confusion during classroom use.

### Blueprint structure and readability

The Blueprint architecture is functional but has evolved over time through iterative development. Some logic could be structured more clearly, with improved separation of responsibilities and cleaner communication patterns between Blueprints.

As the project grows, introducing clearer architectural diagrams and refactoring certain interaction chains would improve maintainability and reduce cognitive load for new developers.

### Interaction consistency and UX clarity

Interaction patterns are not fully standardized across exercises. For example, remote grabbing is available in some contexts but not others, and certain actions lack audio or visual feedback.

While the core functionality works, improving interaction consistency, adding clearer affordances, and introducing feedback mechanisms (such as audio confirmation or object movement animation during remote grab) would strengthen usability and reduce ambiguity

during lessons.

## Rendering and asset structure

Dynamic lighting is currently used to support flexible level restructuring. While this allows fast iteration, it introduces additional rendering cost compared to baked lighting. A future transition to baked lighting, once the environment stabilizes, would likely improve performance.

Similarly, environmental assets are not yet fully consolidated or optimized for minimal draw calls. Shader consolidation, improved LOD coverage, and environmental simplification represent opportunities for refinement. LOD usage is generally not yet fully standardized across assets. Some meshes lack optimized LOD chains, and consolidating environmental assets could reduce draw calls and replication overhead in multiplayer sessions.

Distance culling is actively used, but other visibility optimization methods such as more systematic use of precomputed visibility volumes have not yet been fully implemented. Expanding these techniques may improve scalability as the number of exercises and environmental elements grows.

Known performance drains **-** Some performance-sensitive areas have been identified during development. Large numbers of small actor-based objects can introduce overhead and pixel overdraw or overused occlusion culling.

Instance-heavy environmental structures and complex Blueprint interaction chains may also affect performance if not carefully managed. These areas are functional but represent opportunities for further optimization as the project matures.

As mentioned post-processing effects are currently kept minimal to maintain flexibility and simplicity over graphical polish.

Stronger settings of anti aliasing can improve edge smoothing but may negatively affect text readability and performance. However currently the project employs TextRenderActors which are flexible to update but not the most readable as letters are negatively affected by aliasing. Anti-aliasing in contrast makes the texts blurrier and harder to read as well.

A future improvement would be to add material and texture based texts and adding functions to dynamically activate readable texts when the user is within readable distance, as currently the flickering to texts are also caused by small letters being viewed from far away.

Overall, the system is stable and operational, but several areas would benefit from modularization, structural cleanup, and clearer separation between experimental features and core infrastructure. Making these improvements incrementally will strengthen the long-term maintainability and scalability of the project.

# 10. Publishing workflow

This chapter describes how VäXR is built, packaged, and deployed to VR headsets, as well as how the corresponding backend multiplayer server is kept in sync. Because the system consists of both a headset application and a containerized server environment, publishing is not only about uploading an APK - it also requires coordination with the backend infrastructure.

The workflow described below reflects the current production setup and should be followed carefully to avoid version mismatches.

## Building the client application

The headset application is built from Unreal Engine as an Android package (APK) with an associated OBB file. The Android build target must be selected in Unreal, and both files are generated during the packaging process. Remember to update the store version in project settings. Without this Meta Quest (or similar publishing tools) will deny the new version as it is considered a duplicate of the previous published version.

Before publishing a new version, all changes should be committed and pushed to the repository to ensure that the codebase reflects the release state. The APK and OBB files are then used for distribution through the Meta publishing tools.

## Backend synchronization before release

Publishing a new client version requires that the multiplayer backend is rebuilt to match the updated logic. When changes are pushed to the production branch, an automated CI/CD pipeline builds a new Docker image of the Unreal dedicated server.

This process typically takes 15–20 minutes. Only after the backend container has been successfully rebuilt should the new client version be considered ready for release. Since there is currently no automatic version validation between client and server, publishing a client without a matching backend can lead to unstable multiplayer behavior.

## Meta Quest Developer Hub and release channels

Distribution to headsets is handled through the Meta Quest Developer Hub, which is connected to the Meta developer portal.

The workflow is as follows:

1. Select the appropriate release channel (Production or Beta/Invite).
2. Upload the APK and OBB files.
3. Enter release notes.
4. Submit the build for validation.
5. Wait for Meta’s automated checks to complete.

Once approved, the application becomes available to headsets subscribed to that specific release channel.

Production releases are distributed to all users with the app installed from the store. Beta or invite-only channels require users to accept an invitation link while logged into their Meta account.

## Release channels and version management

The system supports multiple release channels (e.g., Production and Beta). A headset can switch between channels, which can sometimes cause confusion if users unknowingly run different versions of the application.

If multiplayer behavior appears inconsistent, it is important to verify that all headsets are running the same release channel and version.

## Permissions and post-update considerations

Application updates may reset system-level permissions on the headset. In particular, microphone permissions must be manually enabled in the Quest system settings. If permissions are not granted, voice communication will not function, and there is currently no in-app notification indicating this.

It is also important to verify headset volume levels after updates, as there is minimal ambient audio in the application and low or muted audio may therefore not be discovered until voice communication is attempted.

## Publishing sequence summary

A safe publishing sequence typically follows this order:

1. Update store version in project settings.
2. Commit and push code changes.
3. Ensure backend container rebuild has completed.
4. Build APK + OBB in Unreal.
5. Upload to Meta Quest Developer Hub.
6. Verify correct release channel.
7. Test multiplayer session after deployment.

Because the system consists of both client and server components, publishing is not complete until both sides are verified to function together.

The publishing workflow is stable but requires careful coordination. Maintaining discipline in version control, backend synchronization, and release channel management is essential to ensure smooth classroom deployment.

# 11. Contacts and sources

This chapter provides information about project ownership, responsible organizations, and where to turn for support, further information, or collaboration. VäXR is the result of cooperation between research institutions, municipalities, and educational partners, and long-term sustainability depends on clearly defined responsibility and communication channels.

This chapter is also intended to make responsibility clear and ensure that future developers, schools, or collaborators know where to seek support and how the project is structured organizationally.

## Project ownership and ongoing responsibility

The ongoing development, maintenance, and operational health of VäXR is coordinated through Sambruk. Sambruk is responsible for managing the service as part of its broader digital school development portfolio.

Schools that are connected to Sambruk and subscribe to its services may gain access to VäXR as part of their digital learning infrastructure. Support, coordination, and service-related questions are handled through Sambruk's established support channels. This ensures that schools receive both technical access and organizational guidance within an existing support framework.

Project on GitHub: https://github.com/Petter-Wannerberg/VaXR

## Primary and secondary contacts

For general inquiries, onboarding, access, or support related to VäXR, the primary contact is Sambruk.

Secondary contacts include:

- **Sandra Wissting**, Project Manager, RISE (sandra.wissting@ri.se)
- **Petter Wannerberg**, Researcher and Developer, RISE (petter.wannerberg@ri.se)

For questions specifically related to classroom implementation and practical school use, **Lèv Grünberg,** Kungsbacka kommun, can provide insight based on hands-on experience integrating the application into teaching practice.

## Participating organizations and contributors

The VäXR project has been developed and piloted in collaboration with multiple organizations:

- **RISE Research Institutes of Sweden**
- **Flens kommun**
- **Göteborgs Stad**
- **Montessori Mondial (Academedia)**

- **Kungsbacka kommun**

These organizations have contributed in different ways, including research integration, development, testing, and classroom deployment.

This work (the project in its entirety) is open source and available on GitHub. One of Its continued developers and maintainers will be the Sambruk organization.

### Personal mentions

- **Lèv Grünberg**, Förskola & grundskola, Kungsbacka kommun: Idea and project initiator.
- **Henrik Kullander**, Primary teacher Presseskolan, Kungsbacka kommun: responsible for pedagogics, testing, evaluation and didactic development.
- **Peter Markkanen**, University Lecturer, Stockholm University: His research in mathematical didactics laid the foundation for the design of the geometry exercises in the project.
- **Petter Wannerberg**, Researcher and VR developer, RISE (Research Institutes of Sweden): Co-creator of the application and responsible for the VR development and design of the application.
- **Jacob Michelsen**, Developer and Researcher, RISE (Research Institutes of Sweden): Co-creator of the application and responsible for system architecture and backend development.
- **Sandra Wissting** – Senior project manager RISE (Research Institutes of Sweden). Responsible for the VäXR project from RISE.s side.

## External libraries, tools, and resources

VäXR builds upon established technical frameworks and third-party tools. Relevant external resources include:

- **Unreal Engine (Epic Games)** – Core development platform
- **VR Expansion Plugin** – VR interaction and multiplayer handling
- **OpenXR** – Cross-platform XR standard
- **Universal Cross-Platform Pro Voice Chat Plugin** – Multiplayer voice integration
- **Docker** – Containerized multiplayer server deployment
- **Parse Server** – Backend database and orchestration layer

For technical documentation and troubleshooting, developers are encouraged to consult the official documentation for Unreal Engine, OpenXR, and the respective plugins in use.

Authors: Petter Wannerberg and Jacob Michelsen.